Universidad Nacional de Colombia
Facultad de Ingeniería

Ejercicios

Presentado por:
Leal Diaz Layonel Camilo
Jaekle Till

Profesor:
José J. Martínez P.

Asignatura:
Inteligencia Artificial y mini-robots

Bogotá, D.C

5 de marzo de 2026

Ejercicio 1. AG_Palabras_JFFG_MEJORADO.ipynb

Resumen de Mejoras del Algoritmo Genético

La refactorización del código inicial transformó un modelo funcional pero inestable en un sistema robusto, rápido y capaz de alcanzar la convergencia con mayor probabilidad. Los cambios clave se agrupan en las siguientes áreas:

Optimización del Rendimiento (Eficiencia): En la versión original, la función de "aptitud" (similitud) se calculaba múltiples veces de forma redundante durante la selección y el ordenamiento. El código fue reestructurado para evaluar a la población una sola vez por generación, lo que aceleró drásticamente el tiempo de ejecución. Además, se eliminó la dependencia de librerías externas (wordfreq), permitiendo que el algoritmo construya individuos desde cero (caracteres aleatorios), un enfoque mucho más afín a la evolución natural y más ligero para el procesador.

Robustez en la Reproducción (Cruce Seguro): Se ajustó la lógica matemática de la función de cruce. Anteriormente, el manejo de cadenas de texto de distintas longitudes podía generar errores de índice o producir cadenas vacías. Ahora, el cruce maneja los cortes de forma dinámica y segura, garantizando descendencia válida en todos los casos.

La Gran Mejora: Mutación Inteligente y Dinámica: Este fue el cambio más decisivo. En las primeras iteraciones, el algoritmo sufría de estancamiento en "óptimos locales" (por ejemplo, quedarse atrapado en "persro" con un 90.9% de similitud durante decenas de generaciones). Esto ocurría porque el modelo original solo podía sustituir letras. La gran innovación fue introducir una mutación capaz de insertar caracteres faltantes, eliminar letras sobrantes o sustituir las incorrectas. Esta flexibilidad le otorgó al algoritmo la capacidad de salir de los estancamientos, logrando converger al 100% de coincidencia con la palabra objetivo en muy pocas generaciones.

Interactividad y Cierre: Se mejoró la retroalimentación del sistema, implementando un bloque final que aísla y reproduce mediante voz únicamente al "cromosoma ganador", permitiendo auditar visual y auditivamente el éxito de la simulación.

Con esto, el código no solo quedó más limpio, sino que conceptualmente demuestra cómo un pequeño ajuste en las reglas de un sistema complejo (la mutación) puede resolver cuellos de botella masivos en su comportamiento general.

Ejercicio 2. agDieta_Mejorado.ipynb

Procedimiento Realizado

Se implementaron dos enfoques computacionales para resolver un problema de optimización de dieta con múltiples restricciones:

Algoritmo Genético (AG) Multi-objetivo: Se utilizó la librería DEAP con el algoritmo NSGA-II para manejar simultáneamente dos funciones objetivo: la minimización del error nutricional (desviación de calorías y macronutrientes) y la minimización del costo total. Se aplicaron operadores de cruce de dos puntos y una mutación personalizada que permite la inserción, sustitución y eliminación de cantidades enteras.

Optimización Matemática Exacta (Gurobi): Se formuló un modelo de Programación Lineal Entera Mixta (MILP) mediante Programación por Metas (Goal Programming). Se definieron variables de decisión enteras y variables de desviación continua para penalizar cualquier incumplimiento de los parámetros nutricionales establecidos.

Resultados Obtenidos

Algoritmo Genético: Logró un costo de $111.35, pero con una desviación nutricional de 1.4234. El conteo calórico final fue de 10,944 kcal, lo que representa un cumplimiento parcial de la meta de 17,500 kcal. El algoritmo tendió a favorecer la reducción de costos sobre la precisión nutricional dentro del frente de Pareto.

Gurobi: Halló la solución óptima global con un costo de $179.45. Logró una precisión prácticamente total en los objetivos nutricionales, alcanzando 17,501 kcal (meta de 17,500 kcal). Las cantidades de productos seleccionadas cumplieron estrictamente con todos los límites mínimos y máximos de stock.

Conclusiones del Algoritmo

Convergencia vs. Exactitud: El solucionador exacto (Gurobi) es superior en este modelo lineal, ya que garantiza la solución óptima global en milisegundos. El algoritmo genético, al ser una metaheurística, proporciona soluciones aproximadas que, en este caso, sacrificaron el cumplimiento de una función objetivo para minimizar la otra.

Sensibilidad de la Función Objetivo: Se determinó que el éxito del Algoritmo Genético depende críticamente de la lógica de selección final en el frente de Pareto. Una ponderación inadecuada entre costo y nutrición permite que el algoritmo converja hacia soluciones económicas pero técnicamente deficientes en términos de salud.

Manejo de Restricciones: Mientras que Gurobi maneja las restricciones como límites rígidos en el espacio de búsqueda, el Algoritmo Genético requiere de penalizaciones externas para evitar que la mutación y el cruce generen individuos que violen los parámetros de inventario.

Ejercicio 3. evolucion_imagenes_mejorado.ipynb

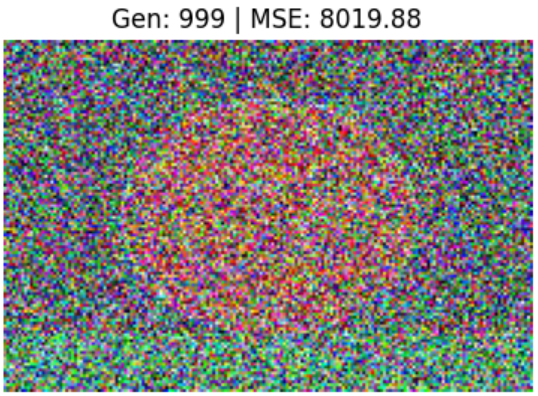
Resumen de Modificaciones Técnicas

Generación Programática del Objetivo: Se eliminó la dependencia de archivos externos mediante la integración de la librería Pillow (ImageDraw). Ahora, el sistema genera automáticamente una "imagen objetivo" basada en formas geométricas y degradados definidos por código, eliminando la necesidad de que el usuario cargue una imagen de referencia.

Visualización Dinámica y Eficiente: Se refactorizó el sistema de visualización utilizando display_handle.update. Esto permite que la evolución del mejor individuo se muestre de forma fluida en una única posición de la pantalla, evitando la generación masiva de gráficas estáticas y reduciendo el consumo de memoria en el entorno de ejecución.

Optimización del Cálculo de Aptitud: El cálculo del Error Cuadrático Medio (MSE) se vectorizó completamente a través de numpy. Esto permite evaluar la población total en una sola operación de matriz, acelerando el procesamiento en comparación con los bucles tradicionales.

Refinamiento de Operadores: Se implementó una mutación aditiva basada en ruido normal. A diferencia de la sustitución aleatoria de píxeles, este método permite cambios graduales en la intensidad del color, facilitando una transición más orgánica hacia los tonos del objetivo.

Conclusión

Aunque se logró un acercamiento visualmente reconocible hacia las formas y colores del objetivo generado, se concluye que un Algoritmo Genético basado exclusivamente en la manipulación individual de píxeles (fuerza bruta de color) presenta una convergencia extremadamente lenta para la complejidad de una imagen. Para este tipo de problemas de reconstrucción visual, el algoritmo podría no ser el más indicado, sugiriendo que enfoques basados en Algoritmos de Composición Geométrica (evolucionar polígonos en lugar de píxeles) o Redes Generativas serían considerablemente más eficientes.